

Super Linearity and the Bigger Machine

Sometimes you can have your scalability and boost it, too

By Herb Sutter

There are two main ways to achieve superlinear scalability, or to use P processors to compute an answer more than P times faster (see Figure 1):

- Do disproportionately less work.
- Harness disproportionately more resources.

Last month [1], we focused on the first point by illustrating parallel search and how it naturally achieves superlinear speedups when matches are not distributed evenly because some workers get "rich" subranges and will find a match faster, which benefits the whole search because we can stop as soon as any worker finds a match.

This month, we'll conclude examining the first point with a few more examples, and then consider how to achieve superlinear speedups by harnessing more resources—quite literally, running on a bigger machine without any change in the hardware.

[Click image to view at full size]

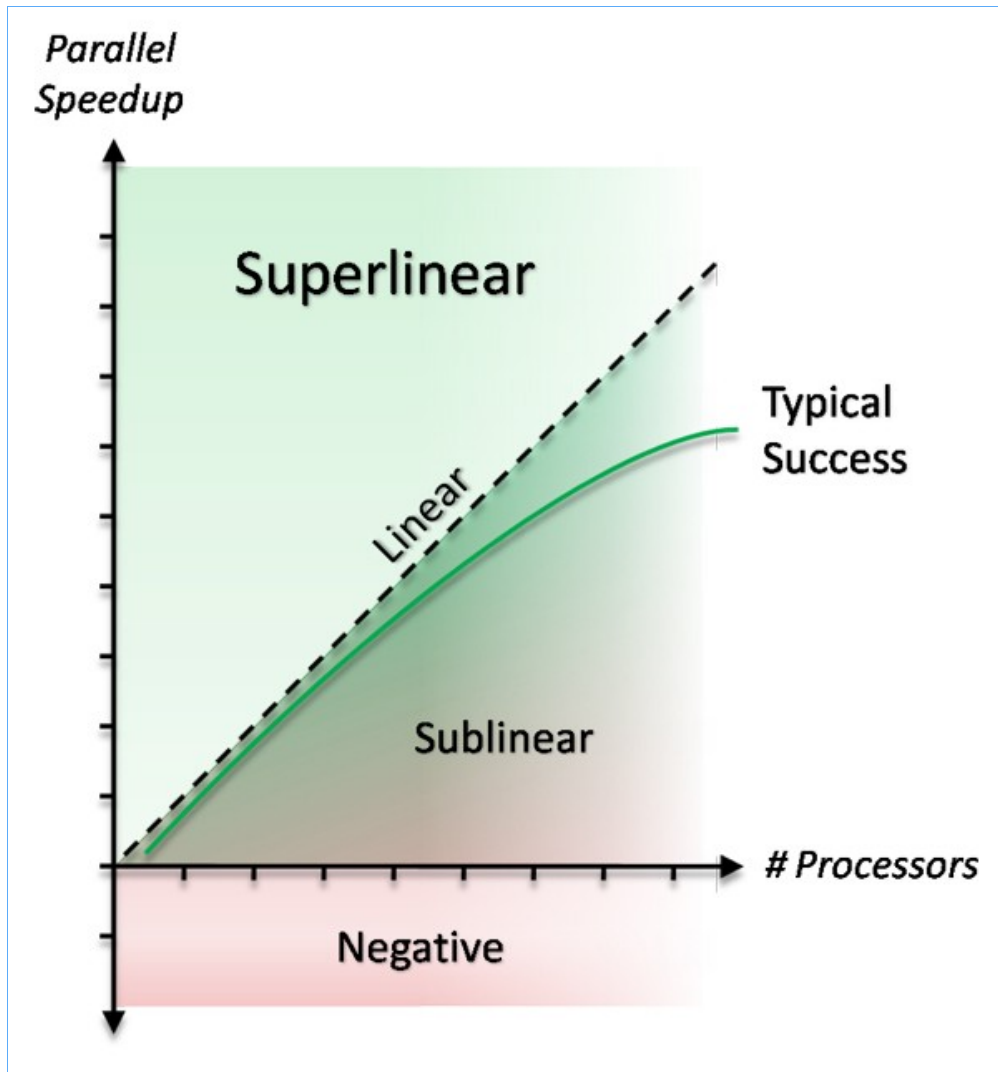


Figure 1: Parallel scalability.

What Have We Learned?

As we saw last month, a parallel search often doesn't need to do anything special to exploit a nonuniform distribution of matches to find a match that much faster; a simple search algorithm will do, such as a linear search for arrays or a depth-first search for trees. Parallel search naturally exploits nonuniform distributions of matches:

- **Without knowing where the nonuniformity is.** It doesn't need any prior locality information, such as location hints for high-probability areas to try first.
- **Without knowing whether there is any nonuniformity at all.** Most interestingly, parallel search naturally benefits from nonuniformity without any extra knowledge at all, not even whether there is any! In this respect, we might say that parallel search is deliciously opportunistic.

Sequential search, on the other hand, cannot exploit nonuniformity at all without advance information about where the high-probability HASH(0x87e42c) are. Having the sequential algorithm visit nodes in a different or randomized order doesn't help by itself without additional information about where to start [2]. Even armed with a map of the high-probability match areas, the sequential algorithm would need to be made more complex to exploit the nonuniformity.

Finally, getting superlinear parallel algorithm performance typically relies on the ability to interrupt or cancel work. There are two aspects to this, one must-have and one good-to-have:

- **Immediate return (necessary).** When a worker finds a match, we must be able to return the result to the caller, and let the caller complete immediately without waiting for other workers to complete normally. Otherwise, parallel performance will still scale, but the scaling will usually be sublinear; the time to complete is the maximum, not the minimum, of all workers, and in our search example, we'd typically have to wait for some unsuccessful workers that take N/P time, which puts the overall search's performance in the same category.
- **Stop abandoned work (highly desirable).** When one worker finds a match, we want to be able to efficiently stop the other workers in order to reclaim their resources, including access to any data they may have locked and the OS and hardware threads they are running on, so that we can use those resources for other work. This is sometimes called "interruption" or "cancellation" and will be the subject of next month's column.

Other Sources of Superlinearity

Another way to achieve superlinear scalability is to exploit partial information. In some parallel algorithms, one worker can discover partial information that it can communicate to other workers to let those others do their work faster than they could do otherwise. For example, in a tree search, one worker might discover that "there are never matches under a node whose weight is over 100." Or, in computing solutions to a set of equations, one worker might discover that in any solution the value of variable x has to be between $y-1$ and $y+4$. Other workers can then use such partial information to run faster by excluding entire sections of their search space—subtrees under heavy nodes, or HASH(0x87e42c) where x and y are too far apart—without having to traverse those areas at all to inspect their contents for possible matches.

Still another potential path to superlinearity is to perform speculative execution. Let's say that you have two alternative algorithms available to compute a result, such as two search algorithms where one has better average-case performance and the other has better worst-case performance. Now, most of the time you're better off just picking one of the algorithms and using available hardware to run it in parallel. But sometimes you can do better by running both of them, and seeing which gets the answer first. The cost is that you're doing more work, but you can get the answer faster (including avoiding the bad worst case of one algorithm). Sometimes the extra work can be relatively "free," at least from your selfish point of view; for example, when the two operations are two database or web service queries against different servers, and one server might get the answer in a different way than the other because the servers may be experiencing different loads or may use different methods to compute the answer.

The bad news about all of the effects we've covered so far is that none of them are going to help you go superlinear with algorithms that have to visit every element in a range anyway, such as when you're calculating the sum of all nodes. The good news, however, is that there is another effect that can help: It's when using more threads lets you use a bigger machine, literally.

Running on a Bigger Machine

Besides leveraging the superlinearity that can crop up naturally in parallel algorithms, how else could we ever achieve a superlinear speedup by using more parallelism on the same machine? After all, more parallelism means we can use more cores to speed up compute-bound work, but just using more cores in itself gives us only a linear speedup. Right?

The fallacy is in the question's final words: "...on the same machine." When you run a program with less parallelism and another with more parallelism on the same modern desktop or server hardware, the one with more parallelism literally runs on a bigger machine—a disproportionately larger share of the available hardware. This happens because the one with more parallelism can use not only additional cores, but additional hardware resources attached to those cores that would not otherwise be available to the program. In particular, using more cores typically also means getting access to more cache and/or more memory.

To see why this is so, consider Figure 2 and Figure 3. Each of these shows a simplified block diagram of the cores and caches on two modern commodity CPUs: the current Intel "Kentsfield" processor, and the upcoming AMD "Barcelona" processor, respectively. The interesting feature in both chips is that e

core has access to cache memory that is not available to some or all of the other cores. In the Kentsfield processor, each pair of cores shares a private L2 cache; in the Barcelona chip, each core has its own private L2 cache. In both cases, no core by itself has access to all the available L2 cache...and that means that code running on just one core is limited, not only to the one core, but also to just a fraction of the available cache. For code whose performance is memory bound, the amount of available cache can make a huge difference.

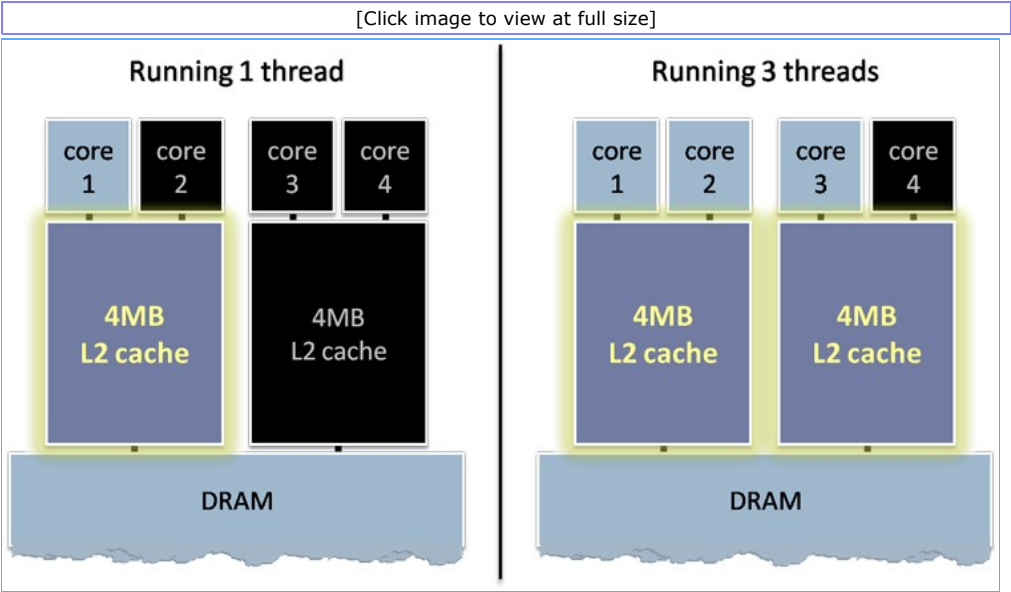


Figure 2: Intel Kentsfield core and cache utilization: 1 thread versus 3 threads.

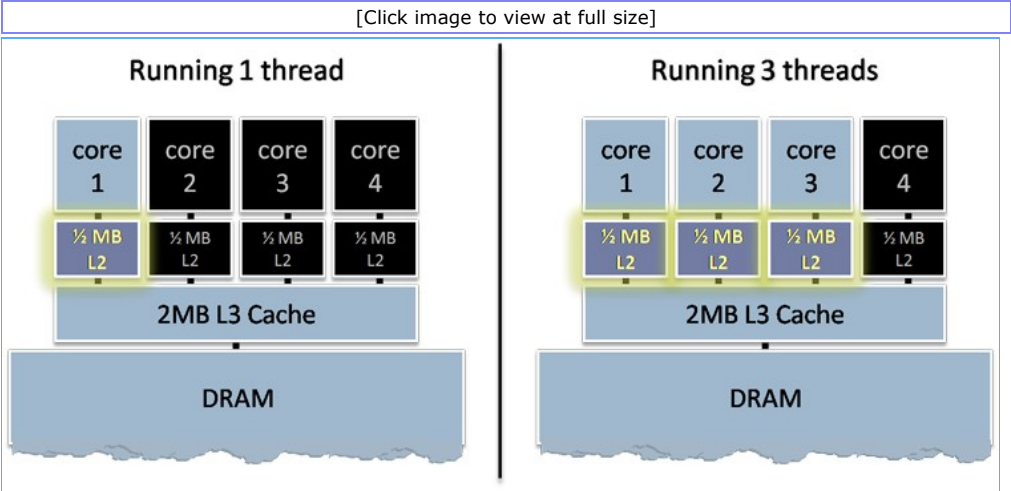


Figure 3: AMD Barcelona core and cache utilization: 1 thread versus 3 threads.

In Figure 2, with only one thread running, as expected we see three of the Kentsfield's four cores are unused. But by running a sequential algorithm we don't only lose three cores; we also lose half the system's available L2 cache. When we run a parallel algorithm using two workers (for example, on core

and 3) or more, not only do we get to use more processors, but we also double the amount of L2 cache available—that is, we literally run on a bigger machine with more cache. Similarly on the Barcelona chip shown in Figure 3, each additional worker adds to the total cache size until we saturate the machine with four workers.

A similar effect occurs on NUMA (nonuniform memory access) machines. Figure 4 shows a sample NUMA architecture where the machine is organized into nodes that each has its own processors and its own RAM. All the processors can use all the RAM in the machine, but the fly in the ointment is that not all RAM is created equal: RAM that's attached to other processors is "further away" and more expensive to use, because it requires a trip through the inter-node interconnect. So this time the question isn't one of making more RAM available, but rather of making faster RAM available.

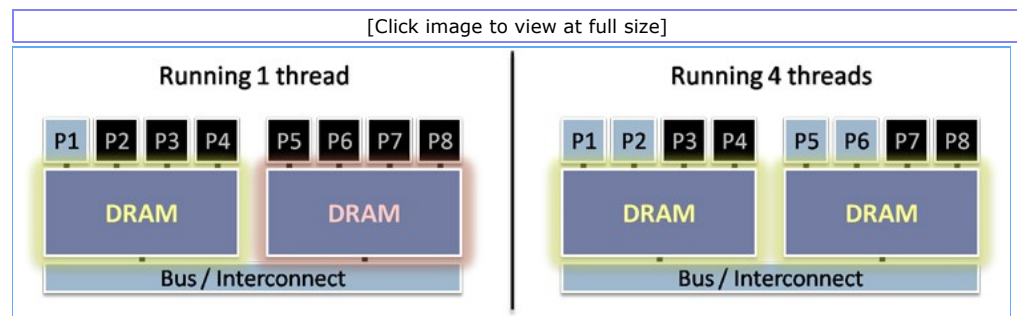


Figure 4: Sample NUMA processor and cache utilization: 1 thread versus 4 threads.

To illustrate, imagine that in Figure 4, the average time to access memory (in the case of a cache miss) is 200 clock cycles if the request goes to directly connected RAM, and 1000 cycles if it's to RAM on another node. A program with only one thread running might experience an average memory access time of around 600 cycles, modulo other cache and locality effects. But a program that runs two or more threads that are spread fairly evenly across the two nodes might experience an average cache miss access time of only 200 cycles. Given that waiting for memory is already a common bottleneck for modern high-performance software, and that we could do an awful lot of computation in those 400 clock cycles if we could use them for something better than sitting stalled while waiting for RAM to respond, getting those cycles back can be a big deal.

Well-written parallel code that is memory-bound and cache-bound can get faster superlinearly to a point because more data fits into memory and cache, and so total computation time goes down because we get more page and cache hits. The more the code makes multiple passes over the data, the more this superlinear effect is amplified.

On Deck

There are two main ways into the superlinear stratosphere:

- Do disproportionately less work.
- Harness disproportionately more resources.

To accomplish the first point, we saw that interruption was essential. But how should we interrupt (or, to use the C-word, "cancel") a thread or a unit of work? More on that when we return...

Notes

[1] H. Sutter. "Going Superlinear" (*Dr. Dobbs's Journal*, March 2008; www.ddj.com/cpp/206100542).

[2] Randomized algorithms do have their place in mitigating worst-case behaviors for sequential code, as noted in this reference: Si. Pi Ravikumar. *Parallel Methods for VLSI Layout Design*, 4.3.2 (Ablex/Greenwood, 1996).